



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 2: PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1 (CLO1).** Phân biệt được yêu cầu chức năng (Functional Requirements) và yêu cầu phi chức năng (Non-functional Requirements) theo chuẩn ISO/IEC 25010:2011.
2. **MT2 (CLO1).** Áp dụng được ít nhất 3 trong 5 kỹ thuật xác định yêu cầu (elicitation): Interview, Observation, Document Analysis, Workshop, Survey.
3. **MT3 (CLO1, CLO3).** Viết được User Story đúng format kèm Acceptance Criteria cho ít nhất 8 yêu cầu của dự án nhóm.
4. **MT4 (CLO2).** Tham gia phân tích nhóm và đóng góp xây dựng danh sách yêu cầu chung cho dự án.

NỘI DUNG

1. Định nghĩa yêu cầu phần mềm
2. Phân loại yêu cầu: Functional vs Non-functional
3. 8 thuộc tính chất lượng theo ISO/IEC 25010
4. Cách viết yêu cầu - User Story và Acceptance Criteria
5. Ưu tiên hóa yêu cầu (MoSCoW)
6. 5 kỹ thuật xác định yêu cầu (elicitation)
7. Khảo sát hiện trạng
8. Anti-patterns (Sai lầm thường gặp)

1. Định nghĩa yêu cầu phần mềm

Định nghĩa “yêu cầu phần mềm” (software requirement) theo tiêu chuẩn quốc tế **ISO/IEC/IEEE 29148:2018**:

Yêu cầu (requirement) là:

- ✓ (a) một phát biểu mô tả nhu cầu hoặc kỳ vọng cùng các ràng buộc và điều kiện; hoặc
- ✓ (b) một biểu diễn được tài liệu hóa của một nhu cầu, khả năng, đặc điểm, ràng buộc, hoặc chất lượng của một hệ thống nhằm đáp ứng giá trị cho một bên liên quan.

1. Định nghĩa yêu cầu phần mềm

Diễn giải đơn giản: Yêu cầu là sự kết hợp của 3 thành phần:

- 1. Nhu cầu (need):** Khách hàng/người dùng cần hệ thống làm gì hoặc đạt mức chất lượng nào.
- 2. Ràng buộc (constraint):** Giới hạn hoặc điều kiện đi kèm (vd: phải dùng SQL Server vì công ty đã mua license).
- 3. Tiêu chí xác minh (verification criteria):** Cách kiểm chứng yêu cầu đã được đáp ứng (vd: thời gian phản hồi < 2 giây).

1. Định nghĩa yêu cầu phần mềm

Một yêu cầu tốt phải có:

- 1. Rõ ràng (Unambiguous):** Mọi người đọc đều hiểu giống nhau. “Hệ thống phải nhanh” → KHÔNG rõ ràng. “Thời gian phản hồi tra cứu sách dưới 3 giây” → rõ ràng.
- 2. Có thể kiểm chứng (Verifiable):** Có cách đo lường để xác nhận yêu cầu đã được đáp ứng. “Hệ thống phải thân thiện” → không kiểm chứng được. “95% người dùng hoàn thành đặt món trong dưới 4 click” → kiểm chứng được.
- 3. Có thể truy vết (Traceable):** Mỗi yêu cầu phải có nguồn gốc rõ ràng (ai đề xuất, từ đâu) và được liên kết đến design, code, test sau này.

1. Định nghĩa yêu cầu phần mềm

Ví dụ: Phần mềm quản lý quán cà phê

- ❑ Yêu cầu “xấu” (vi phạm 3 đặc tính): “Hệ thống cần dễ dùng và nhanh.”
- ❑ Yêu cầu “tốt” (cùng ý đó nhưng đã refine):
 - ✓ “Thu ngân có thể nhập đơn 5 món trong dưới 30 giây bằng thao tác chuột hoặc bàn phím” (đo được, kiểm chứng được).
 - ✓ “Hệ thống phản hồi mọi thao tác cập nhật trong dưới 2 giây ở môi trường wifi tối thiểu 5 Mbps” (rõ ràng, kiểm chứng được).

2. Phân loại yêu cầu

1. **Functional Requirement (FR) – Yêu cầu chức năng**
2. **Non-functional Requirement (NFR) - Yêu cầu phi chức năng**

2. Phân loại yêu cầu

Functional Requirement (FR) – Yêu cầu chức năng:

- ✓ Định nghĩa theo Sommerville (2016): “Functional requirements are statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.”
- ✓ Diễn giải: **FR** mô tả hệ thống **LÀM GÌ**, các chức năng cụ thể, hành vi đầu vào/đầu ra.

2. Phân loại yêu cầu

Functional Requirement (FR) – Yêu cầu chức năng:

Đặc điểm của FR:

- ✓ Mô tả một hành động cụ thể của hệ thống.
- ✓ Có thể test bằng phương pháp đúng/sai (pass/fail).
- ✓ Thường có động từ chính: “Hệ thống cho phép...”, “Hệ thống tính...”, “Hệ thống lưu...”.

2. Phân loại yêu cầu

Functional Requirement (FR) – Yêu cầu chức năng:

□ Ví dụ FR cho Quán cà phê:

- ✓ **FR-1:** Hệ thống cho phép thu ngân tạo đơn hàng mới.
- ✓ **FR-2:** Hệ thống tính tiền đơn theo công thức: $\text{tổng} = \sum(\text{số lượng} \times \text{đơn giá}) - \text{giảm giá}$.
- ✓ **FR-3:** Hệ thống áp dụng giảm giá 10% tự động cho khách có thẻ thành viên.
- ✓ **FR-4:** Hệ thống xuất báo cáo doanh thu hàng ngày dạng PDF.

2. Phân loại yêu cầu

Non-functional Requirement (NFR) - Yêu cầu phi chức năng

- ✓ Định nghĩa theo Sommerville (2016): “Non-functional requirements are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards.”
- ✓ Diễn giải: NFR mô tả hệ thống **LÀM NHƯ THẾ NÀO** - các ràng buộc về chất lượng, hiệu năng, an toàn, khả dụng...

2. Phân loại yêu cầu

Non-functional Requirement (NFR) - Yêu cầu phi chức năng

Đặc điểm của NFR:

- ✓ Liên quan đến chất lượng (quality attribute), không phải hành động.
- ✓ Test bằng đo lường định lượng (số liệu) hoặc đánh giá chất lượng.
- ✓ Thường có tính từ và số liệu cụ thể: “trong 2 giây”, “99.9% uptime”.

2. Phân loại yêu cầu

Non-functional Requirement (NFR) - Yêu cầu phi chức năng

❑ Ví dụ NFR cho Quán cà phê:

- ✓ **NFR-1** (Performance): Thời gian tính tiền một đơn (≤ 10 món) dưới 1 giây.
- ✓ **NFR-2** (Reliability): Hệ thống phải hoạt động được khi mất internet (offline mode), tự đồng bộ khi có lại mạng.
- ✓ **NFR-3** (Usability): Nhân viên mới có thể sử dụng thành thạo sau dưới 30 phút training.
- ✓ **NFR-4** (Security): Mật khẩu nhân viên mã hóa bằng bcrypt với cost factor ≥ 10 .

3. 8 thuộc tính chất lượng theo ISO/IEC 25010

- ✓ Khi viết một Non-functional Requirement cụ thể, ta gắn nó với một trong 8 thuộc tính chất lượng dưới đây. **Mỗi NFR thuộc về một thuộc tính. Một thuộc tính có thể có nhiều NFR.**
- ✓ **Ví dụ:** thuộc tính Performance Efficiency có thể có 3 NFR khác nhau cho cùng một hệ thống (về thời gian phản hồi, throughput, sử dụng RAM).

3. 8 thuộc tính chất lượng theo ISO/IEC 25010

Theo **ISO 25010**, chất lượng phần mềm gồm 8 thuộc tính (characteristics):

#	Thuộc tính	Ý nghĩa	Ví dụ NFR cho Quán cà phê
1	Functional Suitability (Phù hợp chức năng)	Hệ thống làm đúng chức năng cần làm	100% chức năng tính tiền đúng theo quy định giảm giá hiện hành
2	Performance Efficiency (Hiệu năng)	Tốc độ, tài nguyên sử dụng	Thời gian tính tiền < 1 giây
3	Compatibility (Tương thích)	Hoạt động với phần mềm/thiết bị khác	Tương thích máy in nhiệt Xprinter, Citizen
4	Usability (Khả dụng)	Dễ học, dễ dùng, dễ nhớ	Nhân viên mới sử dụng thành thạo dưới 30 phút

3. 8 thuộc tính chất lượng theo ISO/IEC 25010

Theo **ISO 25010**, chất lượng phần mềm gồm 8 thuộc tính (characteristics):

#	Thuộc tính	Ý nghĩa	Ví dụ NFR cho Quán cà phê
5	Reliability (Độ tin cậy)	Tính sẵn sàng, chịu lỗi, phục hồi	Uptime $\geq 99\%$ trong giờ mở quán; offline mode khi mất net
6	Security (Bảo mật)	Bảo vệ dữ liệu, xác thực, phân quyền	Mật khẩu mã hóa bcrypt; phân quyền theo role
7	Maintainability (Khả năng bảo trì)	Dễ sửa, dễ test, dễ thay đổi	Code có unit test coverage $\geq 70\%$
8	Portability (Khả chuyển)	Cài được trên nhiều môi trường	Chạy được trên Windows 10, 11; web hoạt động trên Chrome, Firefox, Safari

4. Cách viết yêu cầu - User Story và Acceptance Criteria

- ✓ Có nhiều cách viết yêu cầu: văn bản tự do, biểu mẫu cấu trúc (xem trong giáo trình), mô hình UML...
- ✓ Trong các phương pháp Agile (Scrum, XP, Kanban) hiện đại, cách phổ biến nhất là User Story + Acceptance Criteria, được giới thiệu bởi Mike Cohn (sách “User Stories Applied”, 2004) và đã trở thành chuẩn de-facto của ngành.

4. Cách viết yêu cầu - User Story và Acceptance Criteria

Format User Story

CẤU TRÚC USER STORY

As a <vai trò>,

I want <tính năng>,

so that <lợi ích>.

(Tiếng Việt: “Là một [vai trò], tôi muốn [tính năng], để [lợi ích].”)

4. Cách viết yêu cầu - User Story và Acceptance Criteria

Ví dụ User Story cho Quán cà phê

✓ US-1:

- **As a** thu ngân,
- **I want** áp dụng giảm giá 10% tự động cho khách có thẻ thành viên,
- **so that** không phải tính nhầm và tránh nhầm lẫn khi giờ cao điểm.

✓ US-2:

- **As a** chủ quán,
- **I want** xem báo cáo doanh thu cuối ngày phân theo món,
- **so that** biết món nào bán chạy để nhập thêm nguyên liệu.

4. Cách viết yêu cầu - User Story và Acceptance Criteria

Acceptance Criteria (Tiêu chí nghiệm thu)

- ✓ Mỗi User Story phải đi kèm Acceptance Criteria (AC) - danh sách các tiêu chí cụ thể để xác định khi nào User Story được coi là “hoàn thành”.
- ✓ Format phổ biến nhất: Given-When-Then (BDD format từ Behavior-Driven Development).

FORMAT GIVEN-WHEN-THEN

Given <tiền đề>

When <hành động>

Then <kết quả mong đợi>

4. Cách viết yêu cầu - User Story và Acceptance Criteria

- ✓ **AC cho US-1 (giảm giá thẻ thành viên):**
 - **AC-1.1: Given** khách có thẻ thành viên hợp lệ, **When** thu ngân tạo đơn hàng, **Then** hệ thống tự động trừ 10% trên tổng tiền.
 - **AC-1.2: Given** khách KHÔNG có thẻ thành viên, **When** thu ngân tạo đơn hàng, **Then** hệ thống KHÔNG áp dụng giảm giá tự động.
 - **AC-1.3: Given** thẻ thành viên đã hết hạn, **When** thu ngân nhập mã thẻ, **Then** hệ thống cảnh báo “Thẻ đã hết hạn” và không áp dụng giảm giá.
- ✓ **Mỗi AC = 1 test case sau này. Đây là cách Agile gắn requirement với testing, một yêu cầu được “xong” khi mọi AC đều pass.**

5. Ưu tiên hóa yêu cầu (MoSCoW)

- ✓ Một dự án thật luôn có hơn 100 yêu cầu nhưng nguồn lực chỉ đủ làm 30-50.
- ✓ Câu hỏi: làm cái nào trước? Đây là bài toán prioritization (ưu tiên hóa).
- ✓ Phương pháp phổ biến nhất là MoSCoW, do Dai Clegg phát triển năm 1994 tại Oracle UK, sau đó được hệ thống DSDM (Dynamic Systems Development Method) áp dụng.
- ✓ Hiện nay được dùng rộng rãi trong Agile, Scrum, đặc biệt khi viết Product Backlog.

5. Ưu tiên hóa yêu cầu (MoSCoW)

MoSCoW là viết tắt của 4 cấp ưu tiên (chữ “o” thêm vào để dễ đọc):

Cấp	Ý nghĩa	Định nghĩa	Ví dụ Quán cà phê
M Must have	PHẢI có	Không có thì sản phẩm thất bại / không thể release	Tạo đơn hàng, tính tiền, in/gửi hóa đơn
S Should have	NÊN có	Quan trọng nhưng không critical, có thể release sau	Báo cáo doanh thu, quản lý tồn nguyên liệu
C Could have	CÓ THỂ có	Tốt nếu có, không có cũng được trong release đầu	Khách quen tích điểm, gợi ý món bán kèm
W Won't have	SẼ KHÔNG có	Đã quyết định KHÔNG làm trong scope hiện tại	Đặt món online, app cho khách hàng

5. Ưu tiên hóa yêu cầu (MoSCoW)

Quy tắc tỷ lệ MoSCoW (theo DSDM Consortium)

- ❑ Khi áp dụng MoSCoW cho một dự án/release, tỷ lệ khuyến nghị (theo effort, không phải số lượng):
 - ✓ Must have: $\leq 60\%$ effort
 - ✓ Should have: $\sim 20\%$ effort
 - ✓ Could have: $\sim 20\%$ effort
 - ✓ Won't have: 0% effort (đã loại trừ)
- ❑ Lý do: nếu Must have $> 60\%$, dự án có rủi ro cao không kịp deadline. Phải để dành buffer cho rủi ro qua các Should/Could.

5. Ưu tiên hóa yêu cầu (MoSCoW)

Lưu ý quan trọng

- ✓ Won't have KHÔNG có nghĩa là “không bao giờ làm”. Nó có nghĩa là “không làm trong scope/release HIỆN TẠI”. Có thể làm trong release sau.
- ✓ MoSCoW phải được KHÁCH HÀNG đồng thuận, không phải team development tự quyết. Đây là phần “đàm phán scope”.

6. Kỹ thuật xác định yêu cầu (elicitation)

- ❑ Elicitation là từ tiếng Anh chuyên ngành (gốc Latin: elicere - “khêu gọi, khám phá”).
- ❑ Trong CNPM, elicitation là quá trình khám phá yêu cầu từ stakeholder.

6. Kỹ thuật xác định yêu cầu (elicitation)

Kỹ thuật 1: Interview (Phỏng vấn)

Phổ biến nhất, áp dụng được mọi tình huống. Có 2 dạng:

- ✓ **Structured** (cấu trúc): Có sẵn danh sách câu hỏi, hỏi tuần tự → tốt cho khảo sát rộng nhiều người.
- ✓ **Unstructured** (mở): Đặt câu hỏi mở, theo dòng câu chuyện → tốt cho khám phá sâu.
- ✓ **Semi-structured** (bán cấu trúc): Có khung câu hỏi nhưng linh hoạt → phổ biến nhất trong CNPM.

6. Kỹ thuật xác định yêu cầu (elicitation)

Mẹo phỏng vấn - Funnel Technique (kỹ thuật phễu):

- ✓ Bắt đầu bằng câu hỏi **MỞ** và **RỘNG** (“Hãy kể tôi nghe về một ngày làm việc bình thường của chị.”)
- ✓ Dẫn thu hẹp về câu hỏi cụ thể (“Khi khách hàng đặt món bằng giọng nói, chị xử lý thế nào?”)
- ✓ Kết thúc bằng câu hỏi xác nhận (“Vậy nếu hệ thống tự động ghi nhận đặt món bằng giọng nói, chị có thấy hữu ích không?”)

6. Kỹ thuật xác định yêu cầu (elicitation)

Kỹ thuật 2: Observation (Quan sát)

Trực tiếp quan sát người dùng làm việc tại nơi làm việc thật của họ. Mạnh nhất khi người dùng “không biết cách diễn đạt” công việc của mình bằng lời.

- ✓ **Passive observation:** Quan sát từ xa, không can thiệp.
- ✓ **Active observation:** Hỏi xen kẽ trong khi quan sát.
- ✓ **Apprenticing:** Tự làm thử công việc của user (thường áp dụng cho UX research.)

6. Kỹ thuật xác định yêu cầu (elicitation)

Kỹ thuật 3: Document Analysis (Phân tích tài liệu)

- ❑ Đọc và phân tích các tài liệu nghiệp vụ hiện có: form, biểu mẫu, báo cáo, quy trình SOP, hợp đồng, báo cáo lỗi cũ, email, biên bản họp...
- ❑ Mạnh khi:
 - ✓ dự án thay thế hệ thống cũ;
 - ✓ domain phức tạp với quy định pháp lý;
 - ✓ cần hiểu nghiệp vụ trước khi gặp khách.

6. Kỹ thuật xác định yêu cầu (elicitation)

Kỹ thuật 4: Workshop / JAD Session (Hội thảo)

- ✓ **Joint Application Design (JAD):** kỹ thuật do IBM phát triển những năm 1970, vẫn hiệu quả đến nay. Tổ chức một buổi họp kéo dài 2-8 giờ với đầy đủ stakeholder (khách hàng + dev + sponsor + BA + designer) để cùng nhau thảo luận và đồng thuận về yêu cầu.
- ✓ **Ưu điểm:** Tốc độ. Vài buổi workshop có thể tương đương vài tuần phỏng vấn lẻ. Đạt đồng thuận nhanh khi có nhiều stakeholder không đồng ý nhau.
- ✓ **Nhược điểm:** Khó tổ chức (lịch của tất cả mọi người); cần facilitator giỏi.

6. Kỹ thuật xác định yêu cầu (elicitation)

Kỹ thuật 5: Survey / Questionnaire (Khảo sát phiếu hỏi)

- ❑ Gửi bảng hỏi (online: Google Forms, Typeform, SurveyMonkey) đến số đông để thu thập dữ liệu định lượng.
- ❑ Mạnh khi:
 - ✓ cần ý kiến của >50 người (vd: end users của một app);
 - ✓ muốn benchmark mức độ ưu tiên giữa các tính năng;
 - ✓ trong tổ chức lớn không thể phỏng vấn tất cả.
- ❑ Yếu khi: cần khám phá sâu. Phiếu hỏi không đào sâu được như phỏng vấn.

6. Kỹ thuật xác định yêu cầu (elicitation)

KHI NÀO DÙNG KỸ THUẬT NÀO?

- ❑ Một dự án thật thường KẾT HỢP nhiều kỹ thuật, không chỉ 1:
 - ✓ Bắt đầu bằng Document Analysis để hiểu domain.
 - ✓ Tiếp theo Interview với 3-5 stakeholder chính để khám phá sâu.
 - ✓ Observation để xác nhận thông tin từ phỏng vấn (workflow thực tế thường khác lời kể).
 - ✓ Workshop để đồng thuận giữa các stakeholder không đồng ý.
 - ✓ Survey để xác nhận với số đông trước khi prioritize.

7. Khảo sát hiện trạng

Khảo sát hiện trạng là một bước cụ thể trong elicitation, nhằm hiểu hệ thống/quy trình HIỆN TẠI của khách hàng trước khi đề xuất hệ thống mới.

Quy trình khảo sát hiện trạng:

- ❑ **Bước 1.** Tìm hiểu tổng quan về tổ chức (2-4 giờ)
 - ✓ Quy mô: bao nhiêu nhân viên? bao nhiêu chi nhánh?
 - ✓ Lĩnh vực hoạt động chính.
 - ✓ Sản phẩm/dịch vụ chính.
 - ✓ Đối thủ cạnh tranh.

7. Khảo sát hiện trạng

Quy trình khảo sát hiện trạng:

❑ **Bước 2. Tìm hiểu cơ cấu tổ chức (2-4 giờ)**

- ✓ Sơ đồ tổ chức (org chart).
- ✓ Trách nhiệm và quyền hạn từng vị trí.
- ✓ Bộ phận nào sẽ sử dụng phần mềm?
- ✓ Đối nội (giữa các phòng ban) và Đối ngoại (với khách hàng, nhà cung cấp).

7. Khảo sát hiện trạng

Quy trình khảo sát hiện trạng:

- ❑ **Bước 3. Tìm hiểu nghiệp vụ chi tiết (8-40 giờ - chiếm phần lớn thời gian)**
- ❑ Với mỗi vị trí công việc liên quan, lập danh sách công việc và thông tin chi tiết:
 - ✓ Đầu vào: Người đó nhận thông tin gì? Từ ai?
 - ✓ Quá trình xử lý: Họ làm gì với thông tin đó? Theo quy tắc nào?
 - ✓ Đầu ra: Họ tạo ra gì? Cho ai?
 - ✓ Phân loại nghiệp vụ: Lưu trữ / Tra cứu / Tính toán / Tổng hợp.

7. Khảo sát hiện trạng

Ví dụ áp dụng vào Quán cà phê (Bước 3 cho vị trí Thu ngân):

- ✓ **Đầu vào:** Khách order (qua lời nói hoặc menu).
- ✓ **Xử lý:** Ghi món vào sổ → tính tiền (máy tính bỏ túi) → thu tiền/cà thẻ → đưa hoá đơn (nếu khách yêu cầu).
- ✓ **Đầu ra:** Hoá đơn cho khách + ticket cho barista pha.
- ✓ **Loại nghiệp vụ:** Lưu trữ (sổ giao dịch) + Tính toán (tổng tiền).

8. Anti-patterns - Sai lầm thường gặp

Anti-pattern 1: Yêu cầu mơ hồ (Ambiguous Requirement)

- ✓ **Sai lầm:** dùng tính từ không định lượng. “Hệ thống phải nhanh”, “giao diện phải đẹp”, “phải dễ dùng”.
- ✓ **Cách khắc phục:** thay tính từ bằng số liệu cụ thể. “Phản hồi dưới 2 giây với 100 user đồng thời.”

8. Anti-patterns - Sai lầm thường gặp

Anti-pattern 2: Yêu cầu kỹ thuật trá hình (Solution-driven Requirement)

- ✓ **Sai lầm:** viết yêu cầu theo cách mô tả CÔNG NGHỆ thay vì NHU CẦU.
“Hệ thống phải dùng MongoDB”, “phải có microservice architecture”.
- ✓ **Vấn đề:** Đây là quyết định kỹ thuật của team, không phải yêu cầu của khách hàng. Khách hàng KHÔNG quan tâm dùng MongoDB hay MySQL, họ quan tâm hệ thống có lưu được dữ liệu khách của họ hay không.
- ✓ **Cách khắc phục:** Hỏi “Tại sao?”. “Tại sao phải dùng MongoDB?” → “Vì cần lưu dữ liệu lớn.” → “Lớn là bao nhiêu? Bao nhiêu transaction/ngày?” → đó mới là yêu cầu thật.

8. Anti-patterns - Sai lầm thường gặp

Anti-pattern 3: Yêu cầu trộn nhiều ý (Compound Requirement)

- ✓ **Sai lầm:** một yêu cầu chứa nhiều ý độc lập. “Hệ thống cho phép thu ngân tạo đơn, in hóa đơn, gửi email cho khách, và cập nhật tồn kho.”
- ✓ **Vấn đề:** Khó test, khó ưu tiên, khó ước lượng effort. Mỗi yêu cầu phải atomic (1 ý duy nhất).
- ✓ **Cách khắc phục:** Tách thành 4 yêu cầu riêng.

8. Anti-patterns - Sai lầm thường gặp

Anti-pattern 4: Yêu cầu không có chủ thể (Implicit Actor)

- ✓ **Sai lầm:** “Hệ thống tự động làm X” hoặc “X được tự động xử lý”.
- ✓ **Vấn đề:** Không rõ AI kích hoạt hành động này, KHI NÀO kích hoạt.
- ✓ **Cách sửa:** ghi rõ trigger. “Mỗi ngày lúc 23:00, hệ thống tự động tạo báo cáo doanh thu và gửi email cho chủ quán.”

8. Anti-patterns - Sai lầm thường gặp

Anti-pattern 5: Yêu cầu cá biệt hóa (Wishful Thinking)

- ✓ **Sai lầm:** yêu cầu nghe rất hay nhưng không khả thi với nguồn lực. “AI tự nhận diện món khách order qua giọng nói tiếng Việt với độ chính xác 99%.”
- ✓ **Cách khắc phục:** cùng khách hàng review tính khả thi, đưa vào Won't have nếu không khả thi với scope.

THỰC HÀNH

1. **Phân loại 24 yêu cầu (làm cá nhân):** Phân loại 24 yêu cầu thành F/NF/X
2. **Workshop khảo sát:** Mỗi nhóm khảo sát đề tài của mình bằng 3 kỹ thuật (Mỗi nhóm chia thành 2 nhóm nhỏ để đóng vai)
3. **Mỗi nhóm viết 12-16 User Story + AC + MoSCoW priority cho dự án** (đề tài cuối kỳ đã đăng ký)

BÀI TẬP VỀ NHÀ

- ❑ **Mục tiêu:** Hoàn thiện và mở rộng danh sách yêu cầu lên ≥ 15 yêu cầu, sẵn sàng để vẽ Use Case Diagram ở buổi sau.
- ❑ **Yêu cầu cụ thể:**
 - ✓ Mở rộng danh sách yêu cầu từ ≥ 26 yêu cầu.
 - ✓ Tỷ lệ: ≥ 15 Functional + ≥ 8 Non-functional (gồm ít nhất 4 thuộc tính ISO khác nhau).
 - ✓ Mỗi yêu cầu có User Story + ≥ 2 Acceptance Criteria + MoSCoW priority.
 - ✓ Lập một bảng tổng hợp Persona - vẽ lại Persona khách hàng, kèm 3 Goals và 3 Pain Points cụ thể (tham khảo template Persona).
 - ✓ Nộp file Excel danh sách yêu cầu + file Word/PDF Persona lên thư mục Drive lớp trước buổi học tiếp theo.